Hidden History of Neural Architecture Search: From 1980s to Today

Discover the untold roots, evolution, and future of NAS

Chapter 1: Origins of NAS: The 1980s Genetic Dawn

Hello friends today we will talk about the real roots of neural architecture search. Many people think it started in the year twenty seventeen. But that is not true at all. In the early nineteen eighties Miller and his partners used genetic algorithms. At that time they designed the neural network topology automatically. In two thousand two a method named Neat came into existence. This method evolved both the network architecture and weights together. It was very different from today's modern approaches. Between two thousand thirteen and sixteen Bayesian optimization found great vision models. Researchers like Bergstra and Mendoza did amazing work back then. However the real modern boom came from Zoph and Le in two thousand seventeen. Their famous reinforcement learning experiment used eight hundred gpus. That experiment ran for about twenty eight days continuously. In total it consumed more than twenty two thousand gpu hours. This massive compute usage is why it grabbed global media attention. Early rnn based controllers generated the architecture like a simple text

string. This idea looked very simple but it was computationally very heavy. Later Efficient Nas came and claimed a thousand times speedup using weight sharing. But multiple later papers showed it was often no better than random search. In that method the controller hidden state almost completely collapsed. Understanding these early roots is crucial before we move to modern challenges.

Now let us talk about the big reinforcement learning boom. In two thousand seventeen, the paper by Zoph and Le was published. It grabbed the attention of the entire world. A massive reinforcement learning model was working behind it. That experiment used eight hundred GPUs. And they ran continuously for about twenty-eight days. This means that a total of more than twenty-two thousand GPU hours were spent. Everyone was shocked to see such huge computing power. This is the exact reason why it got so much media attention. The early reinforcement learning controllers worked in a very interesting way. They were based on RNNs, which means recurrent neural networks. These controllers generated the design of new neural networks like a long string. This idea felt very simple when you heard or saw it. But in reality, this method was computationally catastrophic. It

wasted a huge amount of computer resources. To solve this big problem, ENAS, meaning Efficient NAS, came later. ENAS claimed that it worked one thousand times faster by sharing weights. But after some time, many research papers came out. They clearly showed that the performance of ENAS was not that good. It often lost even to simple random search and weight sharing. The main reason for this was that the hidden state of the controller almost completely collapsed. This means the model failed completely from the inside. Because of this, early reinforcement learning based methods proved very expensive. Still, this technique brought a new revolution in the world of AutoML. After this, people realized that good models could be built even without human help. However, the difficulties in this path were very high in the early days. Both the computing cost and time became very big hurdles. Despite this, the foundation of today's modern AI systems rests on this reinforcement learning boom.

Chapter 2: Evolutionary NAS & AutoML-Zero: Aging Algorithms

Now let us talk about evolutionary algorithms and AutoML-Zero.

In evolutionary NAS, the aging evolution style is very famous.

In the AmoebaNet style, young architectures are preferentially killed off.

This is the core idea behind regularized evolution.

Google introduced AutoML-Zero in the year two thousand twenty.

It evolves machine learning algorithms from pure basic mathematical operations.

Even the architecture itself is evolved from scratch here.

This is an extreme version with almost zero human bias.

Evolutionary methods are naturally better suited for multi-objective optimization.

Accuracy, latency, and energy are optimized all at once.

Maintaining a Pareto front is very easy for these methods.

The NSGA-II algorithm is heavily used for this purpose.

Evolutionary techniques can achieve multiple goals simultaneously.

This advantage makes them special in modern NAS.

Just like fossil records, they learn from older generations.

Models with better results are carried forward.

Weaker models are removed from the path.

It works entirely on the rule of natural selection.

Therefore, it is also called a genetic algorithm.

This technique works on the pattern of nature.

Without human intervention, great models are prepared.

Now let us talk about Differentiable NAS and its flaws. The DARTS family is a very big name in this approach. In DARTS, we use a continuous relaxation method. Here softmax is applied over all the available operations. This method works at a very high speed. But it suffers from a very serious performance collapse problem. DARTS often ends up selecting only the simple skip connections. The architecture parameters of DARTS are not reliable

indicators. They do not show the final discrete architecture performance well. Sometimes operations with high parameters turn out to be weak. DARTS frequently gets trapped inside very sharp minima. The continuous solution looks extremely good during the search process. But after making it discrete, the performance drops drastically. In many variants of DARTS, memory consumption increases heavily. Memory grows linearly with the total number of operations used. This makes training a full supernet extremely difficult for researchers. This is a massive limitation of this particular technique. People consider it fast, but it fails quite often. Many other techniques have also emerged in this research field. But the flaws of DARTS surprised the entire research community. Researchers are still trying hard to solve these deep issues. Now we move forward and talk about Hardware-Aware NAS.

Chapter 3: Hardware-Aware NAS: One Device, One Performance

Now let us talk about hardware aware nas and some surprising facts. A model optimized for one hardware can perform much worse on another hardware. For example a model searched for iphone x becomes slower on samsung s eight and vice versa. proxylessnas was among the first methods to search directly on target hardware without a proxy dataset. it measured latency directly on the real device. once for all trains a single supernet. from this sub networks can be extracted for different hardware constraints like mobile cpu or gpu. hardware cost estimation strategies have two major categories. one uses lookup tables or predictors. the second uses real on device measurement. real measurement is accurate but very slow. multi objective hardware aware nas optimizes more than just accuracy and latency. it includes energy memory footprint and even thermal constraints as objectives. now let us look at some surprising and counter intuitive facts. in many well designed nas search spaces random search is surprisingly competitive. this happens because the search space is already heavily biased by human

knowledge. strong architectures have a very high density in such spaces. unsupervised neural architecture search is also possible. architecture quality can be estimated using pretext tasks like rotation prediction or colorization. you do not need any labels for this task. the rank correlation with supervised accuracy goes up to zero point eight.

Now let us talk about the world beyond reinforcement learning and differentiable search.

Have you ever thought if NAS is possible without any labels.

This is known as unsupervised neural architecture search.

Here the quality of the architecture is not checked by data labels.

Pretext tasks are used for this purpose.

For example coloring or jigsaw puzzles are used.

Great networks are found without using any labels through these methods.

The rank correlation with supervised accuracy is more than zero point eight.

Chapter 4: Label-Free NAS: AutoML-Zero's Self-Evolving Magic

It means NAS can work brilliantly even without labels.

Now let us talk about AutoML-Zero.

Google introduced it in the year two thousand twenty.

It builds machine learning algorithms completely from scratch.

Only basic mathematical operations are used in this.

It is totally free from human thought and bias.

The architecture evolves itself completely from nothing.

This method feels like science fiction but it is true.

The machine itself decides which path it has to choose.

Human intervention here is almost zero.

Evolutionary and genetic algorithms are great for multi-objective optimization.

Latency and energy must be managed along with accuracy.

Handling the pareto front is very easy for them.

NSGA-two is widely used for this purpose.

Only accuracy is not enough in hardware aware NAS.

A model searched for iPhone X can be slow on Samsung S8.

Therefore a different thinking is needed for every hardware.

ProxylessNAS started searching directly on the target hardware.

Once-for-All trains a single supernet.

Sub-networks are extracted from it according to different hardware needs.

Lookup tables are used to estimate the hardware cost.

Some people also measure directly on the device.

Real measurement is very accurate but runs very slowly.

Energy and memory footprint also hold great importance nowadays.

Even thermal constraints are taken care of here.

The world of NAS is truly vast and surprising.

Now let us talk about hardware constraints and some surprising realities. An architecture optimized for one hardware can perform much worse on another. For example a model searched for iPhone X becomes slower on Samsung S8 and vice versa. ProxylessNAS was one of the early methods that searched directly on target hardware without proxy datasets. It measured latency directly on the actual device. Once for All trains a single supernet from which sub networks are extracted at deployment time for different hardware constraints like mobile or CPU or GPU. There are two major categories for hardware cost estimation strategies. The first category uses lookup tables or predictors. The second category uses real on device measurement. Real measurement is accurate but it is very slow. In multi objective hardware NAS we do not just look at accuracy and latency. We also include energy and memory footprint and even thermal constraints. Now let us talk about some facts that will completely surprise you. In many well designed NAS search spaces random search is surprisingly competitive. This sounds very strange but it is true. This happens because the search space is already heavily biased by human knowledge. The density of strong architectures in it is very high. Unsupervised neural architecture search is

also completely possible. The quality of architecture can be estimated using pretext tasks like rotation prediction or colorization or jigsaw puzzle. There is no need for any labels for this. Its rank correlation with supervised accuracy goes up to zero point eight plus. Seeing all this makes us realize that the world of NAS is still very large.

Chapter 5: From 1989 to 2017: NAS’s Rapid Rise

The roots of neural architecture search started in nineteen eighty-nine. Zoph and Le brought a massive boom in twenty seventeen. Evolutionary methods led to genetic algorithms and AutoML-Zero. Darts provided high speed but also showed several severe flaws. Hardware awareness made it very simple for mobile and devices. Finally random search and unsupervised learning surprised everyone. This technology is completely transforming the world of AI today. Our journey ends right here. Thank you all very much for listening.